

Nortek Vector PUV Pipeline

User Guide and Reference

Holden Leslie-Bole
Scripps Institution of Oceanography

Revised July 2026

Abstract

This is the entry point for anyone using the Nortek Vector PUV pipeline. It takes raw pressure-velocity (PUV) instrument files through five processing levels into quality-controlled time series, wave spectra, and nearshore wave-forcing diagnostics, using one deployment-agnostic configuration system. Sections 2–5 are a step-by-step guide to running your own deployment. Section 6 documents the quality-control and data-provenance system, including the per-channel QC that recovers storm-peak velocity from partial sensor failures. Sections 7–9 are the methods and output reference. You do not need to read it front to back: to just run the code, read §2–5 and §6.

Contents

1	What the pipeline is	3
2	Getting started	3
2.1	Requirements	3
2.2	First run	4
3	Configuring a deployment	4
3.1	Adding a new deployment	4
3.2	Quality-control options: <code>cfg.qc0pts</code>	5
4	Raw-binary ingestion and the pre-2019 archive	5
4.1	Choosing the ingest path	5
4.2	Two things the firmware-1.21 recorders get wrong	6
4.3	Robustness the archive forced	6
4.4	Split recorder files	6
5	Running the pipeline, level by level	7
6	Quality control and data provenance	7
6.1	Why channel decoupling	7
6.2	Per-channel validity	8
6.3	The temperature discriminant	8
6.4	Sound-speed correction	8
6.5	Provenance flags	9
6.6	Failure signatures	9

7	Methods	9
7.1	L1: quality control and coordinate rotation	9
7.2	L2: spectral analysis	10
7.3	L3: wave forcing characterization	11
8	Validation	11
9	Output reference	12
10	Troubleshooting and where to look next	12

1 What the pipeline is

Nearshore wave and current dynamics are measured with bottom-mounted Nortek Vector acoustic Doppler velocimeters (ADV) in pressure-velocity mode, sampling continuously at 2 Hz. The pipeline turns the raw instrument files into standardized outputs at five levels. It ingests either form of Nortek raw data — the ExploreV ASCII export (`.dat/.sen/.hdr`) or the raw recorder binary (`.VEC/.vec/.049`), which it decodes itself (§4); no Windows tooling is required. Every deployment runs through the same drivers and produces the same output structures, so results are comparable across sites and years. The catalog currently spans 26 deployments and 45 instrument-records, from a 2015 Cardiff winter through the 2025 Torrey and SIO Pier seasons.

Level	Output	What it contains
L1	QC'd 2 Hz time series	Burst merge, clock-drift and tilt correction, per-channel quality control, rotation to a geographic frame
L2	Per-segment spectra	Multi-taper power spectra, pressure correction to surface elevation, bulk wave parameters (H_s , T_p , direction), near-bed velocity, bed and Reynolds stress, velocity moments
L3	Forcing metrics	Frequency-band energy decomposition, storm detection, transport proxies (Shields, Rouse), tidal and undertow current decomposition
L4	Nonlinear / IG diagnostics	Surface-elevation bands, incident/reflected infragravity split, bispectra (skewness, asymmetry, bicoherence), velocity PDFs
L5	<i>(planned)</i>	PUV–altimeter integration: bed change vs. forcing

L1–L3 are universal. L4 adds nonlinear-wave and infragravity diagnostics on top of L2. Each level reads the previous level's `.mat` files, so you always run L1→L2→L3→L4 in order.

2 Getting started

2.1 Requirements

- **MATLAB** R2023b or newer, with the **Signal Processing Toolbox** (multi-taper and Welch spectra) and the **Aerospace Toolbox** (`igrfmagm`, the IGRF magnetic-declination model used in the L1 heading rotation). `t_tide` must be on the path for the L3 tidal analysis. The Mapping Toolbox is *not* required (a common mix-up — `igrfmagm` is an Aerospace function), and neither is the Statistics and Machine Learning Toolbox (skewness, kurtosis, and correlation are computed inline in L1–L4; only a few optional `validation/` scripts use it).
- **Internet at runtime**: L2 fetches the shore-normal angle from the CDIP THREDDS server (it falls back to buoy coordinates if unavailable).
- **Raw data**: point each deployment's `cfg.rawDataRoot` at wherever the raw Nortek files live; a local folder is fine, no lab server is required. The bundled San Diego configs read from the SIO group share `/Volumes/group/PUV_data/Vector/`, with deployment metadata in `/Volumes/group/DeploymentNotes/`.

2.2 First run

```
% 1. From the repo root, put every pipeline directory on the MATLAB path:
cd /path/to/PUV_Pipeline
startup_puv

% 2. (Recommended) cache the raw files locally -- network reads are slow, and the
%   drivers automatically use raw_cache/ when it exists:
reg = deployment_registry();
cfg = reg('TBR23')();
copy_raw_to_local(cfg);

% 3. Run the levels in order. Set the deployment name at the top of each driver:
PUV_L1_driver    % raw -> QC'd timeseries      (outputs/L1/TBR23/)
PUV_L2_driver    % spectra + bulk params        (outputs/L2/TBR23/)
PUV_L3_driver    % forcing characterization     (outputs/L3/TBR23/)
PUV_L4_driver    % nonlinear / IG dynamics      (outputs/L4/TBR23/)
```

To process every registered deployment, use the `*_run_all` batch scripts (`PUV_L1_run_all`, `PUV_L2_run_all`, ...), which skip deployments that already have outputs. Outputs land in `outputs/L{1..4}/{deployment}/{label}` and are not version-controlled; they are reproducible from raw data plus code.

3 Configuring a deployment

Every deployment is registered in `config/deployment_registry.m`, which maps a short name to a config function — for example `'TBR23' → @TBR23_config`. The naming convention is `SITE + year + optional season letter` (`TBR23` = Torrey Pines summer 2023; `TOR24S` = Torrey spring 2024; `SI024A--C` = SIO Pier 2024 chronological). For the pre-2019 seasons, which ran one instrument at a fixed station and swapped it every 5–6 weeks, the trailing letter is instead the deployment *ordinal* within that winter (`TOR15A` = first Torrey deployment of 2015–16).

3.1 Adding a new deployment

1. Copy an existing `config/<SITE>_config.m` as a template.
2. Fill in the processing parameters from the authoritative source, `/Volumes/group/DeploymentNotes/DeploymentNotes`, per-instrument `latlon`, `heading`, `clockDrift`, and `doffp` (pressure-sensor height above the bed). The annual checkout spreadsheets have serial numbers but not these numerical parameters.
3. Register the deployment in `config/deployment_registry.m`.
4. Check `config/CONFIG_REVIEW_NOTES.md` and `config/DOFFP_LOOKUP_CHECKLIST.md` for deployment-specific gotchas.

For a deployment ingested from raw binary (§4), also set `instr.rawFormat = 'VEC'`, and for a firmware-1.21 recorder with a wrong clock epoch add `instr.clockSource = 'filename'`, `instr.deployYear`, and `cfg.qcOpts.cutoffGapSec = 60`. See `config/TorreyOffshore_config.m` for a worked example. Metadata for the older seasons lives in `PandPUV2015–2025.xlsx` (the season-by-season PUV inventory) and the `VectorPUV_Winter201*Checkout.xlsx` sheets rather than the `DeploymentNotes` files.

3.2 Quality-control options: `cfg.qcOpts`

The L1 quality control has one option struct, `cfg.qcOpts`, with sensible defaults. You only need to set it deliberately in one case, but that case matters (see the boxed note). The fields:

Field	Default	Meaning
<code>corrMin</code>	70	Minimum beam correlation (%), Nortek’s recommendation.
<code>Tvalid</code>	[-2 40]	Plausible water-temperature range (°C). The primary thermistor-failure discriminant. See the boxed note.
<code>maxJump</code>	Inf	Optional per-sample temperature-step gate; Inf = off. Catches instantaneous glitches, never genuine drift.
<code>tiltStdMax</code>	2	Rolling-std tilt-variability threshold (°).
<code>tiltAbsMax</code>	30	Absolute tilt = toppled frame; always invalidates velocity.
<code>tiltWindow</code>	120	Samples for the rolling std (60 s at 2 Hz).
<code>cFactorTol</code>	0.002	Below this, no sound-speed rescale is applied.
<code>cutoffGapSec</code>	1	Gap between consecutive sensor records that counts as a battery cutoff (s). Raise it (e.g. 60) for the firmware-1.21 recorders, which leave a benign few-second gap in nearly every hourly file (§4).

Set `Tvalid` to the site’s water-temperature range. The default [-2 40] is a wide safety bound that catches gross failures (freezing, firmware garbage) anywhere on Earth, but it treats an implausibly cold in-water reading from a failing thermistor as valid. For San Diego coastal deployments set `cfg.qcOpts.Tvalid = [9 26]`; this catches the subtler within-bounds thermistor failures (a December reading of 8 °C at Torrey Pines is not real water) without ever mis-flagging genuine seasonal or upwelling temperatures. Section 6 explains why.

4 Raw-binary ingestion and the pre-2019 archive

Most deployments before 2019 exist only as raw Nortek recorder binary (`.VEC/.vec/.049`), with no ExploreV ASCII export — and where an export does exist it can be partial (one deployment’s `.dat` covers 5 days of a 174-day record and ends mid-line). The pipeline therefore decodes the binary directly with `read_VEC.m`, so ExploreV — and Windows — are never needed. The decoder is verified bit-exact against the one ASCII export that overlaps it: over 886,319 velocity and 443,131 system records, every integer-valued column (velocity counts, amplitude, correlation, clock, flags) is identical, and pressure and attitude agree to the ASCII’s print precision (`test_read_VEC.m`).

4.1 Choosing the ingest path

Set `instr.rawFormat` to `'VEC'` or `'ASCII'` to force a path; left unset, L1 auto-detects and prefers the ASCII export when a `.dat` is present. *That preference is a trap for an interrupted export*, so L1 warns whenever both forms exist and points at `rawFormat = 'VEC'`; pin it whenever the ASCII might be short. Sampling rate and coordinate system come from the binary’s User Configuration

record, so a deployment with a 0-byte `.hdr` — which cannot be read at all from ASCII — still ingests.

4.2 Two things the firmware-1.21 recorders get wrong

The 2015–2018 instruments run firmware 1.21 and need two corrections, both handled automatically once the config opts in:

Sample rate. The User Configuration field implies 8 Hz on this firmware, but the records are unambiguously 2 Hz. `read_VEC` measures the rate from the decoded records (the ratio of velocity to 1 Hz system records) rather than trusting the configuration field, and warns when the two disagree. This is confirmed three independent ways: the decoded record ratio is exactly 2, the system-record count equals the clock span in seconds, and the field checkout sheet states “Sample rate = 2Hz, Samples per block = 7168, Block time = 3600 s”.

Clock epoch. These recorders stamp their first sample with a nonsense epoch (2000-01-01 or 2002-01-01). The clock is not dead — it *runs* correctly, just from the wrong start — and the recorder names each hourly file `MMDDHHMM` for the true wall-clock hour. A single constant offset reconciles the two, recovered by `vec_clock_from_filenames.m`: opt in with `instr.clockSource = 'filename'` and `instr.deployYear`. The offset is the median over every file and is rejected (on the 98th percentile, so a few dying-battery outliers do not veto a good deployment) if it is not near-constant, so a wrong filename convention fails loudly rather than silently shifting the record. Every recovered span matches the logged deployment date; the one instrument in the set whose clock *was* set correctly serves as the control, and filename-derived time agrees with it to 33 s.

4.3 Robustness the archive forced

Processing these records surfaced several edge cases the modern continuously-sampled instruments never produced, now handled in L1:

- **Benign hourly gaps.** Firmware 1.21 drops a few seconds in nearly every hourly file. `cfg.qc0pts.cutoffGapSec` (§3.2) raises the battery-cutoff threshold above them (60 s for these configs) so a hiccup is not read as the instrument dying.
- **Power-on glitch.** Some recorders write one sample at power-on, then a multi-minute gap, then the deployment. The leading glitch is trimmed off the front rather than ending the record.
- **Battery-death clock drift.** A dying battery pulls the clock off in the last hours; the record is truncated at that jump, and the affected tail is flagged as having approximate absolute timing (its wave statistics are unaffected).
- **Year-crossing deployments.** `MMDDHHMM` filenames sort January ahead of the previous November, so a deployment that spans New Year is reordered chronologically by its (monotonic) clock before the bursts are merged.

4.4 Split recorder files

The recorder splits large records at 100 MiB with a `_N` suffix, and names hourly files by wall-clock hour; either way each raw file is treated as its own burst, so the battery-cutoff detector never

mistakes a file-boundary seam for a real gap. Files are concatenated in chronological (not lexical) order.

5 Running the pipeline, level by level

Each level has a driver (`PUV.L{n}_driver.m`) with a `deployment_name` variable at the top. Set it and run. The batch versions (`PUV.L{n}_run.all.m`) loop over the registry.

- L1 — raw to QC'd time series.** Reads the raw `.dat/.sen/.hdr`, merges bursts to a continuous 2 Hz series, corrects clock drift, applies per-channel QC (§6), corrects tilt, and rotates velocity to the geographic buoy frame ($+x$ west, $+y$ north, $+z$ up). Writes one `*_processed.mat` per instrument holding a PUV struct with the time series and a PUV.qc provenance sub-struct.
- L2 — spectra and wave statistics.** Segments the L1 series into 1-hour windows, estimates multi-taper spectra, converts pressure to surface elevation, and computes bulk wave parameters, near-bed velocity, bed and Reynolds stress, and velocity moments. Writes an L2 struct, one row per segment. This is the level most analyses read.
- L3 — forcing characterization.** Decomposes energy into frequency bands, detects storm events (with MOP gap-filling), computes transport proxies (Shields, Rouse), and separates tidal from subtidal (wave-driven) currents. Writes an L3 struct.
- L4 — nonlinear / IG dynamics.** Adds surface-elevation band products, an incident/reflected infragravity split, bispectra, and velocity PDFs on top of L2. Writes an L4 struct.

Each driver errors with a clear message if the previous level's output is missing, so run them in sequence. Runtime is dominated by reading raw files over the network; cache locally first (`copy_raw_to_local`).

6 Quality control and data provenance

This is the part that most affects whether your results are trustworthy, so read it even if you skip the methods. The guiding principle is **channel decoupling**: the pressure, velocity, temperature, and tilt channels are judged independently, and one dead channel never discards data from a channel that is still good.

6.1 Why channel decoupling

A Nortek Vector has a Doppler velocity sensor and an auxiliary sensor block (pressure, thermistor, compass). These fail independently. The pipeline's original QC nulled an entire data row whenever *any* channel failed, on the assumption that a compromised instrument gives unreliable velocity. That assumption is often false. At Torrey Pines in December 2023, one frame's pressure sensor and thermistor died during a storm while its Doppler channel stayed healthy (beam correlations 92–97%); the old QC threw away the velocity anyway, including every hour of the largest waves in the deployment. Roughly 120 hours of storm-peak velocity were recoverable and had been discarded.

The current pipeline judges each channel on its own evidence and records what it did, so a partial failure costs only the channel that actually failed.

6.2 Per-channel validity

L1 computes four per-sample validity masks, using the thresholds in `cfg.qcOpts`:

- **Velocity** is valid when the minimum beam correlation is $\geq \text{corrMin}$ and the frame is not toppled (see tilt, below). A dead pressure sensor or thermistor does *not* invalidate velocity.
- **Pressure** is valid inside $[\bar{P}/2, 2\bar{P}]$, where $\bar{P}$ is a reference median from a healthy in-water window.
- **Temperature** is valid inside the physical range `Tvalid`, optionally with a per-sample step gate (`maxJump`).
- **Tilt** is judged two ways. A *variability* spike (rolling std above `tiltStdMax`) is a noisy reading; an *absolute* tilt beyond `tiltAbsMax` (30°) means the frame has toppled or moved.

Tilt is not an auxiliary channel. Pressure and temperature say nothing about the Doppler measurement, so they cannot gate it. Tilt is different: it describes the measurement geometry. A toppled frame contaminates the velocity with its own motion, and no rotation repairs that, so an absolute tilt beyond 30° *always* invalidates velocity — regardless of the state of any other channel. A variability spike is trusted only when the sensor block it lives on is healthy; when the block has failed, the jittery tilt reading is discarded and a static tilt from the healthy part of the record is substituted for the rotation.

6.3 The temperature discriminant

The thermistor matters more than it looks, because the instrument uses the measured sound speed — which it computes from temperature — to scale recorded velocity. A dead thermistor therefore biases every velocity linearly. The discriminant for “failed” is *physical plausibility* (`Tvalid`), not a distance from the record mean. A distance-from-mean test would flag a genuine cold-upwelling event as a failure and corrupt its velocity; a physical-range test never does, because real seawater at a real temperature is always valid. This is why `Tvalid` should be set to the site’s water range (§3.2): the narrower the physically-justified range, the more subtle failures it catches, with no risk to good data.

6.4 Sound-speed correction

Where the thermistor has failed but the velocity is good, the velocity is rescaled to remove the sound-speed bias, following the Nortek manual (N3015-030, §2.4.9):

$$V_{\text{corrected}} = V_{\text{recorded}} \frac{c_{\text{true}}}{c_{\text{measured}}}.$$

The reference sound speed c_{true} comes from a companion frame if the config supplies one (`cfg.soundSpeedRef`), otherwise from the instrument’s own healthy temperature–sound-speed relation, evaluated at a reference temperature and never extrapolated beyond it. Where the thermistor is healthy the factor is exactly 1, so the correction is a no-op on good data — a property, not a threshold.

6.5 Provenance flags

Nothing reconstructed or rescaled is ever presented as a clean measurement. Every L2 segment carries provenance so downstream analyses can filter:

Field	Meaning
<code>segValid</code>	Old-style “all channels good” flag; unchanged meaning, so existing analyses keyed on it behave exactly as before.
<code>segValid_vel</code>	Velocity products (moments, mean flow, Reynolds stress) are usable.
<code>segValid_p</code>	Pressure products (H_s , S_η , depth) are usable.
<code>qc_flag</code>	QARTOD-style: 1 good, 2 not evaluated, 3 suspect (recovered or sound-speed-rescaled), 4 fail (physically implausible). Anything reconstructed is 3, never 1.
<code>Hs_source</code>	<code>'measured'</code> / <code>'reconstructed'</code> / <code>'none'</code> .
<code>vel_c_factor</code> , <code>vel_c_corrected</code>	The sound-speed factor applied (exactly 1 on healthy data) and whether it was applied.

To use clean-only forcing, filter on `qc_flag` = 1. To use the storm-peak velocity moments recovered from a sensor-block failure, take `qc_flag` = 3 with `segValid_vel` true, knowing they carry a few-percent scale uncertainty. A segment whose pressure is present but implausible (inflated depth from a drifting sensor) is flagged `qc_flag` = 4 and contributes to nothing.

At L4 these flags travel per burst as `puv_qc_flag`, `puv_segValid_vel`, and `puv_segValid_p`; `qc_flag` = 4 segments are dropped from every merged field.

6.6 Failure signatures

Three distinct failure modes appear in the archive. A one-pass scan of the `.sen` files classifies them without reading the (much larger) `.dat` files, because the `.sen` carries battery, sound speed, heading, and tilt:

Signature	How to recognize it	Velocity recoverable?
Sensor-block failure	Temperature implausible, sound speed steps, battery erratic; tilt stays small	Yes (with the rescale)
Toppled / moved frame	Temperature, sound speed, battery all normal; tilt grows past 30° or drifts steadily	No
Doppler failure	Beam correlation collapses to ~5–10%, amplitude drops; the auxiliary block looks fine	No

The `.sen` scan cannot see a Doppler failure (beam correlation lives in the `.dat`), so “healthy” in that scan means the sensor block and frame are healthy, not that the data are necessarily good.

7 Methods

7.1 L1: quality control and coordinate rotation

Raw burst files — decoded from ASCII or binary (§4) — are merged into a continuous 2 Hz series in chronological order. Clock drift is corrected linearly over each deployment from the drift mea-

sured at recovery. A battery-cutoff detector flags intermittent sampling by finding gaps beyond `cfg.qcOpts.cutoffGapSec` (default 1 s) between consecutive sensor records and discarding everything after the first such gap; a power-on glitch at the very start of the record is trimmed rather than treated as a cutoff. Per-channel QC (§6) is then applied. Samples with stable but non-zero tilt (e.g. a pipe bent during a storm) are retained and corrected by a sample-by-sample rotation using the measured pitch ϕ and roll θ ,

$$\mathbf{v}_{\text{corrected}} = \mathbf{R}_{\text{roll}}(\theta) \mathbf{R}_{\text{pitch}}(\phi) \mathbf{v}_{\text{raw}},$$

applied before the heading rotation so the velocity is returned to the local vertical frame before projection onto geographic coordinates. Velocity is then rotated from the instrument XYZ frame to the buoy frame (+ x west, + y north, + z up; left-handed, matching the MOP convention) using the deployment heading and the IGRF magnetic declination at the deployment location and epoch. The heading rotation is static (it uses the surveyed heading, not the live compass), so a failed compass never corrupts it.

7.2 L2: spectral analysis

Segmentation. The L1 series is divided into non-overlapping 1-hour segments (7200 samples at 2 Hz), aligned to the top of the UTC hour. The 1-hour length matches the cadence of the CDIP MOP wave hindcast and gives the longer averaging window that segment-mean velocity benefits from (random Doppler error in $\bar{u}$ falls as $1/\sqrt{N}$, so the 7200-sample noise floor is $\approx 1.9\times$ smaller than a 2048-sample 17-minute window). A stationarity audit confirmed that bulk parameters are stable on the 1-hour scale at these sites. Segments with more than 10% NaN in a channel group are dropped; smaller gaps are linearly interpolated. Velocity is rotated to a shore-normal frame (+ x onshore, + y alongshore north) using the shore-normal angle from CDIP for the matching MOP station. Depth is $H = \bar{P} \times 10^4 / (\rho g) + z_s$ with $\rho = 1025 \text{ kg m}^{-3}$, $g = 9.81 \text{ m s}^{-2}$, and z_s the sensor height above the bed.

Spectral estimation. Power spectra use the multi-taper method of Thomson (1982) with time-bandwidth product $NW = 4$ ($K = 7$ Slepian tapers, ≈ 14 degrees of freedom per bin). The 1-hour segment gives a native resolution $\Delta f \approx 2.8 \times 10^{-4} \text{ Hz}$. Pressure–velocity cross-spectra use the same tapers. Multi-taper is chosen for its bias–variance trade-off: at constant record length it matches Welch’s effective bandwidth while preserving fine frequency structure that Welch averaging smears. (Welch remains available as a fallback.)

Pressure correction. The pressure spectrum is converted to surface elevation with the linear-theory transfer function $K_p(f) = \cosh(kz_s) / \cosh(kH)$, where $k(f)$ solves $\omega^2 = gk \tanh(kH)$ by Newton’s method, and $S_\eta = S_p / K_p^2$. Bins where $K_p < 0.1$ are zeroed to suppress high-frequency noise amplification; the resulting cutoff f_{cut} is recorded.

Bulk parameters. Significant wave height is $H_s = 4\sqrt{m_0}$ with $m_0 = \int S_\eta df$, integrated separately over the sea–swell (0.04–0.25 Hz) and infragravity (0.004–0.04 Hz) bands. Peak period is $T_p = 1/f_p$ from the sea–swell spectral peak; $T_{m02} = \sqrt{m_0/m_2}$. Mean direction comes from the first-harmonic coefficients a_1 , b_1 of the pressure–velocity cross-spectra. Energy flux is $F = \rho g \int c_g(f) S_\eta(f) df$.

Near-bed velocity, stress, and moments. Near-bed orbital velocity is obtained by scaling each velocity-spectrum bin by $\cosh(kH)/\cosh[k(H - z_s)]$ and inverse-transforming; U_b (rms) and A_w (orbital excursion) follow. Wave bed shear stress uses the Swart (1974) friction factor with $k_s = 2.5 D_{50}$ ($D_{50} = 0.25$ mm by default, to be replaced with site grain size). Reynolds stresses and turbulent kinetic energy come from the detrended fluctuations. Velocity skewness S_k and asymmetry A_s (Hilbert form) characterize the wave nonlinearity relevant to sediment transport.

7.3 L3: wave forcing characterization

The surface-elevation spectrum is integrated over infragravity (0.004–0.04 Hz), swell (0.04–0.10 Hz), and sea (0.10–0.25 Hz) bands, with a dominant-band flag by energy flux. Storm events are found by threshold exceedance ($H_s > 1.5$ m for ≥ 6 h, events within 12 h merged); to characterize forcing during PUV gaps, hourly MOP model data are loaded for a window extending 7 days past the record, and storms present in MOP but absent from the PUV are flagged as gap-period storms. Transport proxies include the Shields parameter $\theta = \tau_b/[(\rho_s - \rho)gD_{50}]$ with the Soulsby–Whitehouse critical threshold and a mobilization flag, cumulative bottom energy flux, and the Rouse number classifying bedload vs. suspended load. Tidal harmonic analysis (`t_tide`, complex $u + iv$) uses the NOAA Scripps Pier tide prediction as the depth reference ($R = 0.995$ vs. 0.987 for a fit to the noisy PUV pressure). Subtidal residual currents (observed minus tidal) capture wave-driven mean flow, including undertow.

8 Validation

The pipeline has been validated against the CDIP MOP wave model and against an independent prior processing pipeline, and its mean-flow output has been tested against physical theory.

- **Against MOP.** PUV bulk wave heights agree well with MOP spectra shoaled to the instrument depth by linear-theory energy conservation. Residual frequency-dependent biases were traced to their sources (nonlinear shoaling, directional narrowing, bound long waves, and MOP spectral-peak broadening) rather than left as unexplained scatter.
- **Head-to-head cross-validation.** On the legacy Ruby2D cross-shore array, the pipeline reproduces an independent prior pipeline’s bulk parameters at $R^2 = 0.93$ –0.98, confirming the L1–L2 chain end to end.
- **Mean cross-shore flow.** The segment-mean cross-shore velocity was tested across 38 instruments against three hypotheses — real Stokes return flow, a constant calibration offset, and random Doppler noise. The data match the Stokes return-flow prediction in sign, magnitude, and response to averaging-window length, and are incompatible with the offset and noise hypotheses. The segment-mean noise floor (~ 0.24 mm s^{−1} at 1-hour averaging) is well below every reported slope and modulation amplitude.
- **Binary decoder.** `read_VEC` is verified bit-exact against the ASCII export (§4). As an independent end-to-end check, every binary-decoded deployment’s hourly H_s was compared to the CDIP MOP model: the modern 2 Hz records (TOR19W/TOR20W/IB19W) give $R^2 = 0.88$ –0.94 with 1–4 cm bias and near-unit slope, and the older firmware-1.21 records ($R^2 = 0.65$ –0.87, all biases under 9 cm) track the model well given their greater distance from the MOP transect and lower Doppler SNR. The near-zero mean vertical velocity in every decoded record independently confirms the decode and the coordinate rotation.

The deeper validation writeups (`results_validation.tex`, `multitaper_writeup.tex`) carry the full tables, figures, and hypothesis tests.

9 Output reference

Outputs are `.mat` files, one per instrument per level, under `outputs/L{n}/{deployment}/`. The most-used fields:

L1 (PUV struct). `time` (naive-UTC datetime), `fs`, `P` (dBar), `BuoyCoord`.`{U,V,W}` (geographic velocity), `T`, `doffp`, and the PUV.qc provenance sub-struct (`valid_vel`, `valid_p`, `valid_tilt`, `valid_T`, `vel_c_factor`, `vel_c_corrected`, `vel_rotation_static`).

L2 (L2 struct, one row per segment). `time`, `Hs`, `Hs_SS`, `Hs_IG`, `Tp`, `Tm02`, `meanDir`, `Ef`, `depth`, `Ub`, `tau_b`, the vmom sub-struct (`skewness`, `asymmetry`, `u_uabs2`, ...), `uMean/vMean`, and the provenance fields of §6 (`segValid`, `segValid_vel`, `segValid_p`, `qc_flag`, `Hs_source`).

Coordinate conventions. After L1 rotation: $+x$ west, $+y$ north, $+z$ up. After L2 shore-normal rotation: $+x$ onshore, $+y$ alongshore north; onshore flux is $q_x > 0$. Pressure is stored in dBar.

10 Troubleshooting and where to look next

- **A driver errors that the previous level is missing.** Run L1→L4 in order; each reads the level below it.
- **Network reads are slow.** Cache raw files with `copy_raw_to_local(cfg)`; the drivers use `raw_cache/` automatically.
- **A deployment recovers little velocity during a storm.** Check the failure signature (§6.6) from the `.sen`: a sensor-block failure is recoverable, a topple is not. Confirm `cfg.qc0pts.Tvalid` is set to the site range.
- **Recovered (`qc_flag = 3`) data in an analysis.** Decide deliberately: filter to `qc_flag=1` for clean-only, or include `qc_flag=3` knowing the scale uncertainty.

Deeper references: `docs/pipeline_levels.md` (level-by-level detail and output-struct shapes), `PIPELINE_NOTES.md` (design decisions, coordinate conventions, known issues), `docs/OUTSTANDING_channel_decoup` and `docs/L1_sensor_block_failure_2026-07-09.md` (the channel-decoupling work in full), `config/CONFIG_REVIEW` (per-deployment review items).